Homework 1

Autoregressive Models, Diabetes Classification, and CUDA Matrix Multiplication

Student	Configuration
Kavan Siddesh Kumar	SID4 9486 | SEED 9486 | HP_ID 0

1. Autoregressive models

An autoregressive model predicts the next value using values that appeared earlier in the same sequence. An AR(p) time-series model uses the previous p observations: $y_t = c + phi_1 y_{t-1} + ... + phi_p y_{t-p} + error_t$. The coefficients are learned from historical data, while the error term captures variation not explained by those lags.

Examples include forecasting electricity demand from recent demand, predicting tomorrow's temperature from recent weather, estimating future sales from past sales, and generating text one token at a time. Autoregressive models are interpretable and efficient, but generation is sequential, early mistakes can influence later predictions, and classical AR models work best when the series is reasonably stationary.

2. Diabetes neural-network experiment

2.1 Personal configuration

Parameter	Value	Model	Configuration
SID4 / SEED	9486 / 9486	Baseline	[64, 32], LR 0.001, 30 epochs
SLICE	486	Modified (HP_ID 0)	[32], LR 0.001, 30 epochs
CLS_A / CLS_B	6 / 0	Training seeds	9486, 9487, 9488

2.2 Data preparation

The headerless CSV contains 759 observations, eight numeric features, and one binary outcome. The final source-label column was renamed and remapped as Diabetes = 1 - SourceLabel, producing 263 diabetes-positive and 496 diabetes-negative observations. Zeros were treated as missing sentinels only for Glucose (5), BloodPressure (35), SkinThickness (224), Insulin (371), and BMI (11); zeros in Pregnancies, DiabetesPedigreeFunction, and Age were retained as valid scaled values.

One fixed stratified 70/15/15 split with random_state=9486 was reused for every model: 531 training, 114 validation, and 114 test rows. Median imputation and StandardScaler were fitted only on the training subset, then applied to validation and test data. The resulting shared arrays were float32, contained no NaNs, covered all 759 rows without overlap, and prevented preprocessing leakage.

2.3 Exploratory analysis

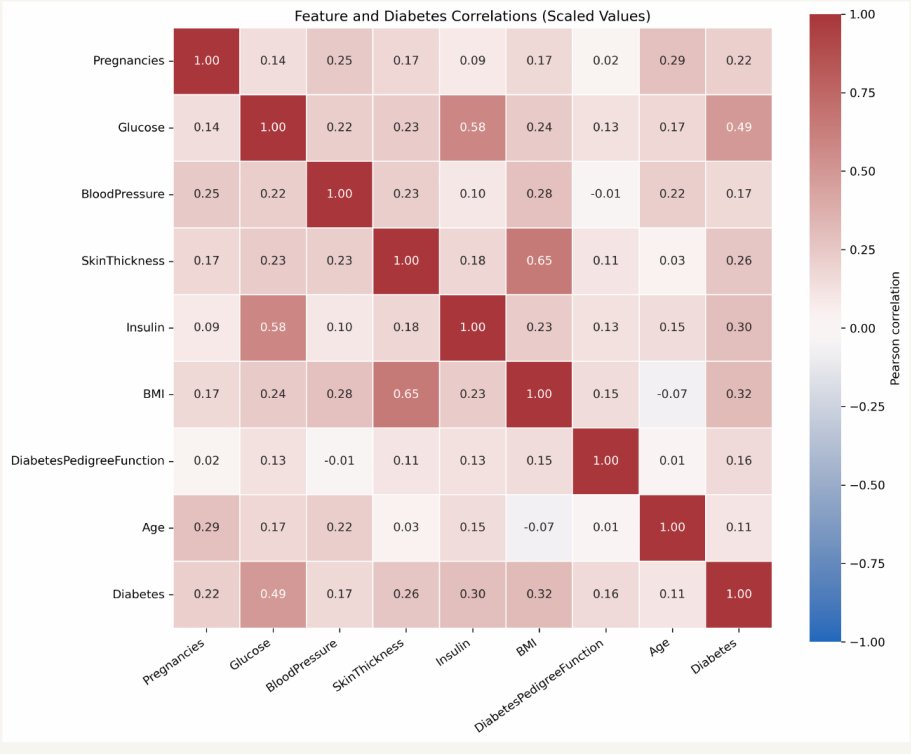

Figure 1. Correlation matrix for the remapped diabetes dataset.

Feature	Correlation	Feature	Correlation
Glucose	0.491538	Pregnancies	0.218405
BMI	0.315051	BloodPressure	0.169221
Insulin	0.303797	DiabetesPedigreeFunction	0.163246
SkinThickness	0.262079	Age	0.112565

Glucose had the strongest positive association with Diabetes (r=0.492), followed by BMI, Insulin, and SkinThickness. All feature correlations were positive and mostly weak to moderate. Insulin and SkinThickness require extra caution because many observations contained missing sentinels. These correlations are descriptive and do not establish causation or statistical significance.

2.3 Exploratory analysis (continued)

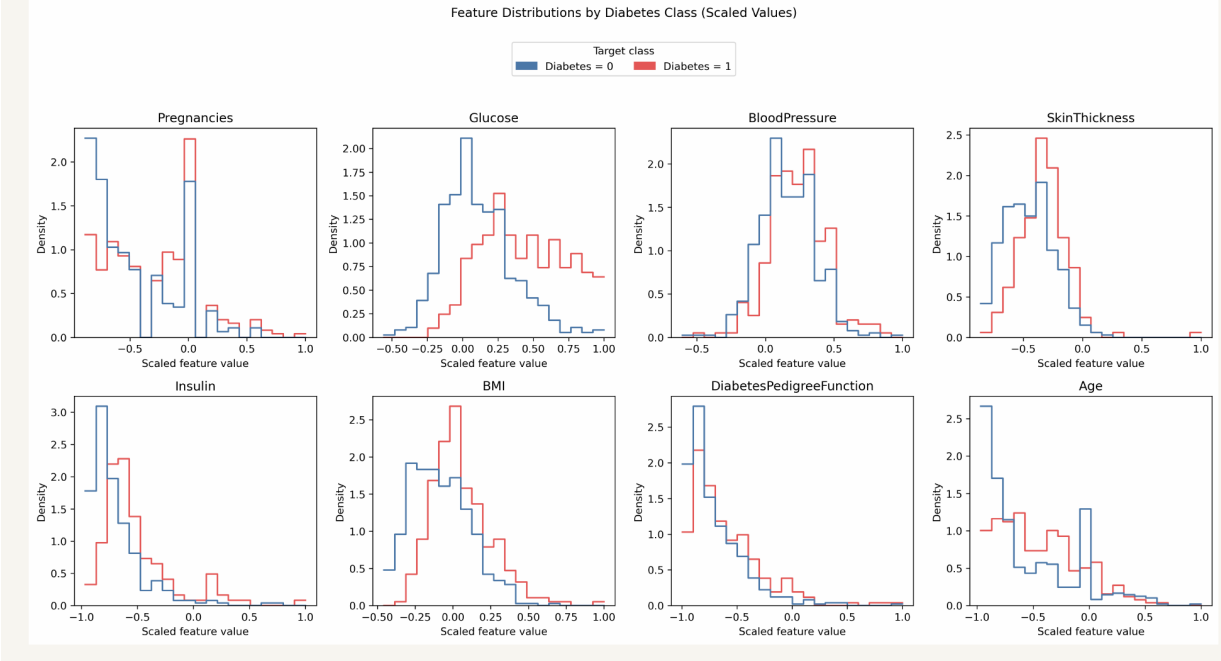

Figure 2. Feature distributions separated by the remapped Diabetes class.

The class distributions overlap for every feature, so no single measurement cleanly separates the outcomes. Glucose and BMI show the clearest shifts, while Insulin and SkinThickness remain affected by extensive missing-sentinel replacement. This supports using a multivariable model while keeping conclusions descriptive.

2.4 Model design and measurement

Model	Hidden layers	Learning rate	Epochs	Parameters
Baseline	[64, 32]	0.001	30	2,689
Modified (HP_ID 0)	[32]	0.001	30	321

Both frameworks used ReLU hidden layers, Adam, batch size 32, the same fixed split and preprocessing, and test accuracy. PyTorch used nn.Module, DataLoader, a manual loop, and BCEWithLogitsLoss; sigmoid was applied only for predictions. TensorFlow/Keras followed the class demonstration with Sequential Dense layers, sigmoid output, binary_crossentropy, model.fit(), model.evaluate(), and model.predict().

Framework	Model	Mean accuracy	Sample SD	Parameters
PyTorch	Baseline	79.24%	0.5064 pp	2,689
PyTorch	Modified	80.41%	1.0129 pp	321
TensorFlow	Baseline	79.53%	3.3210 pp	2,689
TensorFlow	Modified	80.70%	0.8772 pp	321

2.5 PyTorch results

Seed	Baseline accuracy	Modified accuracy
9486	79.8246%	81.5789%
9487	78.9474%	79.8246%
9488	78.9474%	79.8246%
Mean	79.2398%	80.4094%
Sample SD	0.5064 pp	1.0129 pp

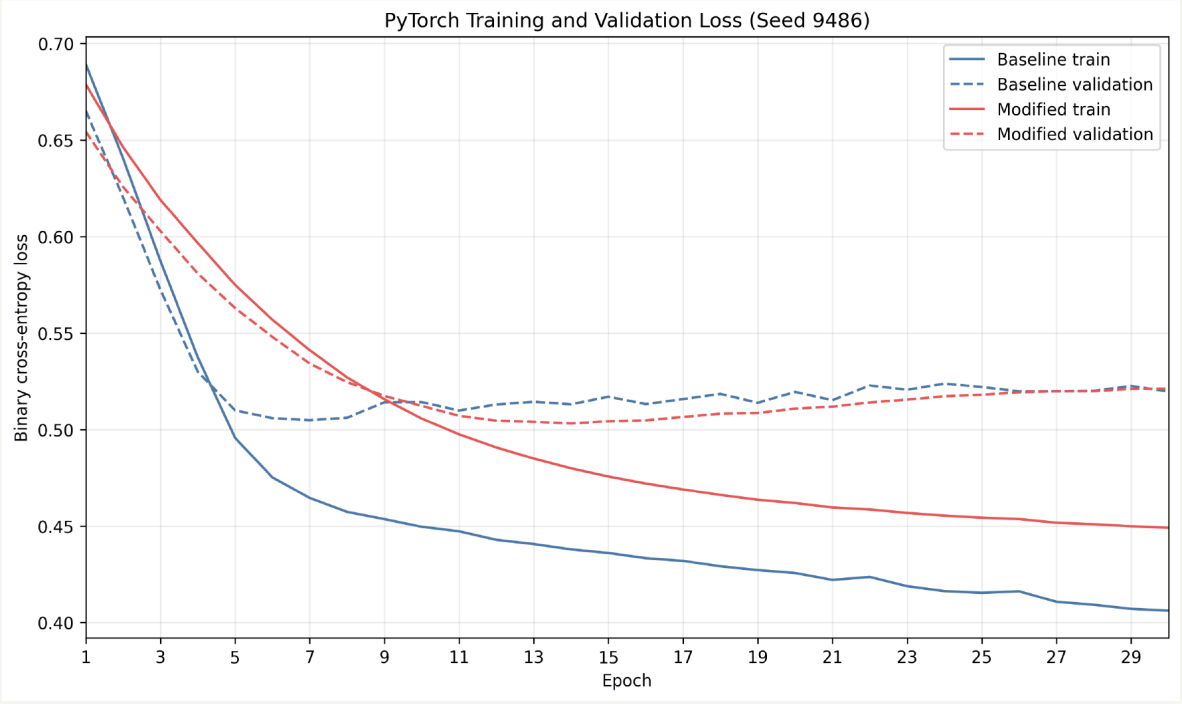

Figure 3. PyTorch training and validation loss for seed 9486.

The baseline reached its minimum validation loss at epoch 7; the modified model reached its minimum at epoch 14. Their final validation-minus-training loss gaps were approximately 0.1135 and 0.0721, respectively. Both curves suggest mild overfitting, but the smaller model's later minimum and smaller gap indicate less separation between training and validation behavior. Mean test accuracy improved by 1.17 percentage points, a modest gain based on only three runs.

2.6 TensorFlow/Keras results

Seed	Baseline accuracy	Modified accuracy
9486	77.1930%	79.8246%
9487	78.0702%	81.5789%
9488	83.3333%	80.7018%
Mean	79.5322%	80.7018%
Sample SD	3.3210 pp	0.8772 pp

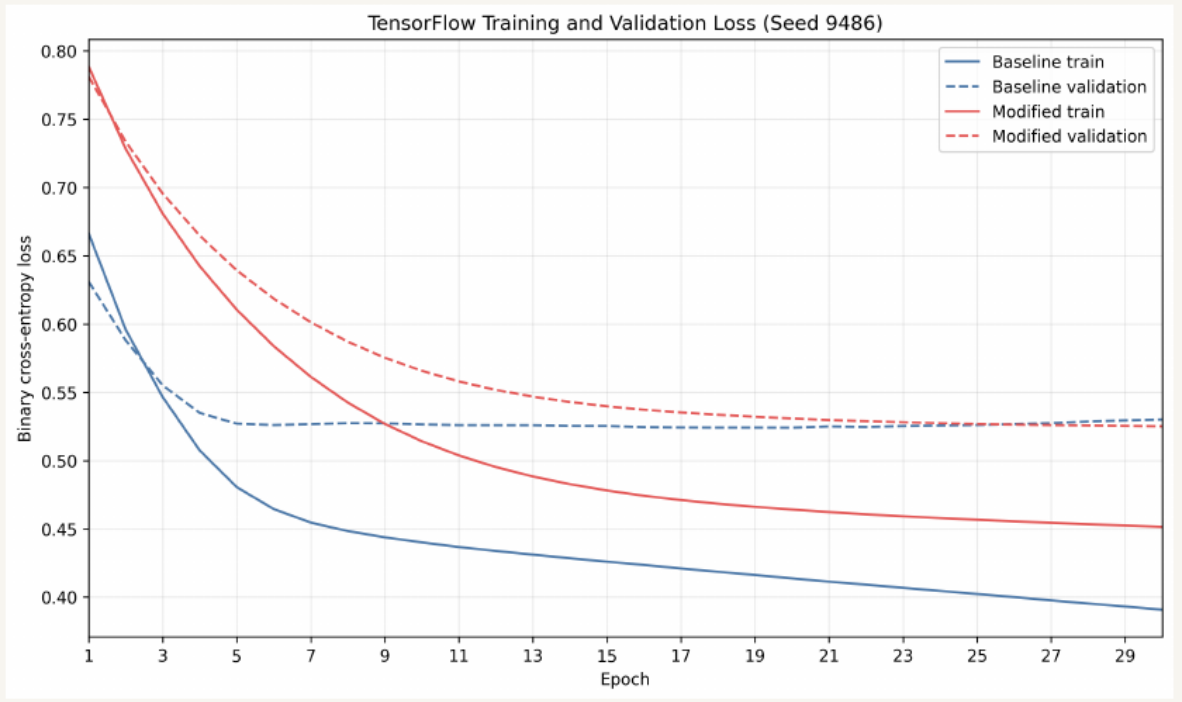

Figure 4. TensorFlow/Keras training and validation loss for seed 9486.

For seed 9486, the baseline validation loss was lowest at epoch 20, with a final validation-training gap of about 0.1394, indicating mild late overfitting. The modified validation loss continued improving through epoch 30 and had a smaller final gap of about 0.0736, so there was no clear validation-loss upturn within the training budget. Mean modified accuracy again improved by 1.17 percentage points and varied less across the three TensorFlow runs.

2.7 Cross-framework comparison

Reducing the network from 2,689 to 321 trainable parameters removed 2,368 parameters (88.1%) while slightly increasing mean test accuracy in both frameworks. TensorFlow's mean was only about 0.29 percentage points higher than PyTorch's for each configuration. With one fixed split and three seeds, these differences do not establish statistical significance or framework superiority; the strongest supported conclusion is that the smaller assigned model was competitive and more parameter-efficient.

3. CUDA matrix multiplication

3.1 Implementation and measurement

The CUDA C program uses a tiled shared-memory kernel with TILE_SIZE=16. Each 16x16 block contains 256 threads, and each thread computes one output element. Tiles outside matrix boundaries are zero-padded and final writes are guarded. OpenBLAS cblas_sgemm provides the CPU reference. After warm-up, the program reports median CPU, kernel, and combined host-to-device plus device-to-host transfer times. Correctness uses the elementwise rule |GPU-CPU| <= 1e-3 + 1e-3|CPU|, avoiding misleading relative-error failures for reference values near zero.

Size	CPU (ms)	GPU kernel (ms)	H2D+D2H (ms)	GPU total (ms)	Speedup
256	0.510085	0.113312	0.258176	0.371488	1.373086x
1024	19.007706	4.370048	2.917920	7.287968	2.608094x
4096	1208.051661	194.599136	54.218529	248.817665	4.855168x

All three sizes passed correctness, and GPU total time includes both kernel and transfer time. GPU execution was beneficial at every tested size; 256x256 was the smallest tested beneficial case. Therefore, these measurements establish only that the crossover occurred at or below 256, not the exact crossover below 256. Launch, allocation, and transfer overhead prevent acceleration from being beneficial at size zero, while larger workloads amortize those fixed costs.

3.2 Scaling and profiling

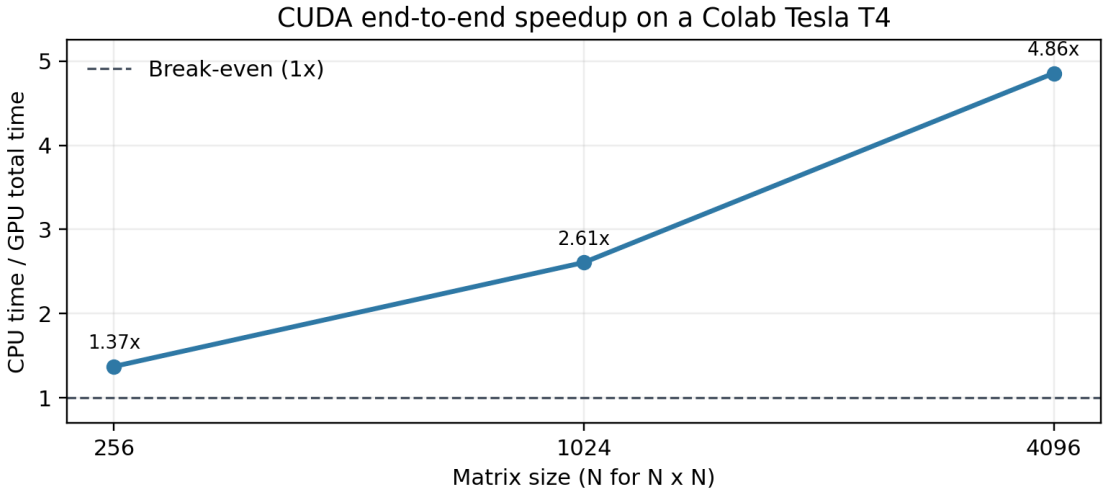

Figure 5. Measured CUDA end-to-end speedup; the dashed line marks break-even performance.

NVIDIA Nsight Compute (ncu) profiled the size-1024 kernel on a Tesla T4 (compute capability 7.5). It reported a 256-thread 16x16 block, a 4,096-block 64x64 grid, 100% theoretical occupancy, approximately 98.7% achieved occupancy, about 74.4% compute and memory throughput, and about 95.3% L1/TEX throughput. Nsight described the kernel as balanced between computation and memory traffic. Individual instrumented kernel durations were approximately 5.79-5.80 ms.

> The program's 1682.78 ms timing under ncu is excluded from the primary table because Nsight replayed and instrumented each launch over nine passes. The performance table instead uses the normal unprofiled CUDA-event kernel time of 4.370048 ms for size 1024.

4. Reproducibility, limitations, and conclusion

Python, NumPy, PyTorch, and TensorFlow seeds were set explicitly; each framework used seeds 9486, 9487, and 9488 on the same fixed split. Executed notebook outputs, six PyTorch checkpoints, six TensorFlow/Keras checkpoints, raw CUDA benchmark output, and successful profiler output are retained in the repository. CUDA timings were measured on a Colab Tesla T4 and may change with hardware, runtime version, thermal state, and session load.

The principal limitations are the small dataset, one fixed train/validation/test split, only three training seeds, and hardware-specific timing. The experiment also did not test CUDA sizes below 256, so it cannot locate the exact crossover. Within those limits, HP_ID 0 reduced model capacity by 88.1% while producing a consistent modest mean-accuracy gain, and the CUDA implementation showed increasing end-to-end benefit as matrix size grew.

AI assistance was used during development and is disclosed separately in AI_USE.md, including specific incorrect suggestions, how they were detected, and the corrections applied.